



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación

Clase 15: Clean Code (Cap. 4)

Francisco Gazitúa Requena (fco_gazitua_requena@uc.cl)

IIC2113 Diseño Detallado de Software

30 de Septiembre, 2025

Comments



“Don’t comment bad code—rewrite it.”

—Brian W. Kernighan and P. J. Plaugher¹

“Nothing can be quite so helpful as a well-placed comment. Nothing can clutter up a module more than frivolous dogmatic comments. Nothing can be quite so damaging as an old crufty comment that propagates lies and misinformation.”

– Robert C. Martin.

“Nothing can be quite so helpful as a well-placed comment. Nothing can clutter up a module more than frivolous dogmatic comments. Nothing can be quite so damaging as an old crufty comment that propagates lies and misinformation.”

– Robert C. Martin.

Resumen del capítulo: El objetivo es que nuestro código esté tan limpio que se entienda sin ningún comentario. Agregar un comentario es un último recurso para esclarecer aspectos poco obvios para el lector.

¿Por qué tanta mala onda con los comentarios?

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

Por otro lado, la única verdad se encuentra en el código (no en lo que un comentario dice que hace un código). Entonces un comentario innecesario hace más mal que bien.

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

Por otro lado, la única verdad se encuentra en el código (no en lo que un comentario dice que hace un código). Entonces un comentario innecesario hace más mal que bien.

Recuerda: *"Comments do not make up for bad code"*

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

Por otro lado, la única verdad se encuentra en el código (no en lo que un comentario dice que hace un código). Entonces un comentario innecesario hace más mal que bien.

Recuerda: *"Comments do not make up for bad code"*

```
1 // Check to see if the employee is eligible for full benefits
2 if ((employee.Flags & HOURLY_FLAG) && (employee.Age > 65))
3     // ...
```

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

Por otro lado, la única verdad se encuentra en el código (no en lo que un comentario dice que hace un código). Entonces un comentario innecesario hace más mal que bien.

Recuerda: *"Comments do not make up for bad code"*

```
1 // Check to see if the employee is eligible for full benefits
2 if (employee.IsEligibleForFullBenefits())
3     // ...
```

¿Por qué tanta mala onda con los comentarios?

- El código cambia pero los comentarios quedan.
- Con el tiempo, los comentarios empiezan a mentir.
- Un comentario que miente puede hacer mucho daño.

Por otro lado, la única verdad se encuentra en el código (no en lo que un comentario dice que hace un código). Entonces un comentario innecesario hace más mal que bien.

Recuerda: *"Comments do not make up for bad code"*

```
1
2 if (employee.IsEligibleForFullBenefits())
3     // ...
```

¿Cuándo un comentario es bueno?

¿Cuándo un comentario es bueno?

Grupo 1:

Comentarios legales, TODOs y documentación de una API pública.

¿Cuándo un comentario es bueno?

Grupo 1:

Comentarios legales, TODOs y documentación de una API pública.

Grupo 2:

Cuando usas una librería externa y hay que clarificar un código que no puedes cambiar.

```
2 // format matched kk:mm:ss EEE, MMM dd, yyyy
3 Pattern timeMatcher = Pattern.compile(
4     "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

¿Cuándo un comentario es bueno?

Grupo 1:

Comentarios legales, TODOs y documentación de una API pública.

Grupo 2:

Cuando usas una librería externa y hay que clarificar un código que no puedes cambiar.

```
2 // format matched kk:mm:ss EEE, MMM dd, yyyy
3 Pattern timeMatcher = Pattern.compile(
4     "\\d*:\\d*:\\d* \\w*, \\w* \\d*, \\d*");
```

Grupo 3:

Explicar la lógica detrás de una decisión o advertir las consecuencias de cambiar algo.

Grupo 3:

Explicar la lógica detrás de una decisión o advertir las consecuencias de cambiar algo.

```
2 //This is our best attempt to get a race condition
3 //by creating large number of threads.
4 for (int i = 0; i < 25000; i++) {
5     WidgetBuilderThread widgetBuilderThread =
6     new WidgetBuilderThread(widgetBuilder, text, parent, failFlag);
7     Thread thread = new Thread(widgetBuilderThread);
8     thread.start();
9 }
```


Grupo 3:

Explicar la lógica detrás de una decisión o advertir las consecuencias de cambiar algo.

```
2 // Don't run unless you
3 // have some time to kill.
4 public void _testWithReallyBigFile(){
5     writeLinesToFile(100000000);
6     response.setBody(testFile);
7     response.readyToSend(this);
8     String responseString = output.toString();
9     assertSubString("Content-Length: 1000000000", responseString);
10    assertTrue(bytesSent > 1000000000);
11 }
```

Grupo 3:

Explicar la lógica detrás de una decisión o advertir las consecuencias de cambiar algo.

```
2 // Don't run unless you
3 // have some time to kill.
4 public void _testWithReallyBigFile(){
5     writeLinesToFile(10000000);
6     response.setBody(testFile);
7     response.readyToSend(this);
8     String responseString = output.toString();
9     assertSubString("Content-Length: 1000000000", responseString);
10    assertTrue(bytesSent > 1000000000);
11 }
```

... y todo comentario que no caiga en estas categorías es (probablemente) malo.

De todos los ejemplos de comentarios malos, uno de los más relevante es el *Commented-Out Code*.

```
2 InputStreamResponse response = new InputStreamResponse();  
3 response.setBody(format.getResultStream(), format.getByteCount());  
4 // InputStream resultsStream = formatter.getResultStream();  
5 // StreamReader reader = new StreamReader(resultsStream);  
6 // response.setContent(reader.read(formatter.getByteCount()));
```

De todos los ejemplos de comentarios malos, uno de los más relevante es el *Commented-Out Code*.

```
2 InputStreamResponse response = new InputStreamResponse();  
3 response.setBody(format.getResultStream(), format.getByteCount());  
4 // InputStream resultsStream = formatter.getResultStream();  
5 // StreamReader reader = new StreamReader(resultsStream);  
6 // response.setContent(reader.read(formatter.getByteCount()));
```

"Few practices are as odious as commenting-out code. Don't do this!"

El tipo de comentario que (para mi) es el más malo es el *Misleading Comment*.

```
1 // Utility method that returns when this.closed is true. Throws an
2 // exception if the timeout is reached.
3 public synchronized void waitForClose(final long timeoutMillis)
4 throws Exception
5 {
6     if(!closed)
7     {
8         wait(timeoutMillis);
9         if(!closed)
10             throw new Exception("MockResponseSender could not be
11             closed");
12     }
13 }
```

Otros tipos de comentarios malos:

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido
- 4 No uses comentarios cuando podrías usar una función o una variable

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido
- 4 No uses comentarios cuando podrías usar una función o una variable
- 5 Atribuciones

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido
- 4 No uses comentarios cuando podrías usar una función o una variable
- 5 Atribuciones
- 6 Comentarios con demasiada información

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido
- 4 No uses comentarios cuando podrías usar una función o una variable
- 5 Atribuciones
- 6 Comentarios con demasiada información
- 7 Tener código que no se usa es igual de malo que tener código comentado

Otros tipos de comentarios malos:

- 1 Escribir comentarios por las puras
- 2 Comentarios redundantes
- 3 Comentarios que solo agregan ruido
- 4 No uses comentarios cuando podrías usar una función o una variable
- 5 Atribuciones
- 6 Comentarios con demasiada información
- 7 Tener código que no se usa es igual de malo que tener código comentado

Recuerda, el mejor comentario es el que encuentras la manera de no escribir

Veamos algunos ejemplos

A veces usamos comentarios para seccionar funciones muy largas

```
2 public static List<string> ProcessNames(List<string> names, char startsWith)
3 {
4     // === 1. Input validation and cleanup ===
5     if (names == null)
6         throw new ArgumentNullException(nameof(names));
7
8     List<string> cleanedNames = new List<string>();
9     foreach (var name in names)
10     {
11         if (!string.IsNullOrEmpty(name))
12             cleanedNames.Add(name.Trim());
13     }
14
15     // === 2. Capitalization of names ===
16     List<string> capitalizedNames = new List<string>();
17     foreach (var name in cleanedNames)
18     {
19         string lower = name.ToLower();
20         string capitalized = char.ToUpper(lower[0]) + lower.Substring(1);
21         capitalizedNames.Add(capitalized);
22     }
23
24     // === 3. Filtering based on starting character (no LINQ) ===
25     List<string> filteredNames = new List<string>();
26     foreach (var name in capitalizedNames)
27     {
28         if (name.Length > 0 && char.ToUpper(name[0]) == char.ToUpper(startsWith))
29             filteredNames.Add(name);
30     }
31
32     return filteredNames;
33 }
```


A veces usamos comentarios para seccionar funciones muy largas

```
2 public static List<string> ProcessNames(List<string> names, char
   startsWith)
3 {
4     // === 1. Input validation and cleanup ===
5     List<string> cleanedNames = GetCleanedAndValidatedNames(names)
6
7     // === 2. Capitalization of names ===
8     List<string> capitalizedNames = GetCapitalizedNames(cleanedNames);
9
10    // === 3. Filtering based on starting character (no LINQ) ===
11    List<string> filteredNames = GetFilteredNames(capitalizedNames,
    startsWith);
12
13    return filteredNames;
14 }
```

A veces usamos comentarios para seccionar funciones muy largas

```
2 public static List<string> ProcessNames(List<string> names, char
   startsWith)
3 {
4
5     List<string> cleanedNames = GetCleanedAndValidatedNames(names)
6
7
8     List<string> capitalizedNames = GetCapitalizedNames(cleanedNames);
9
10
11     List<string> filteredNames = GetFilteredNames(capitalizedNames,
    startsWith);
12
13     return filteredNames;
14 }
```

Ya vimos que tener código comentado está mal

```
2 // public class InvitationEffect : Effect
3 // {
4 //     private GameState _state;
5 //     public InvitationEffect(Game State)
6 //         => _state = state;
7
8 //     public override void Apply()
9 //     {
10 //         SelectTarget();
11 //         SelectPosition();
12 //         // Summon and revive unit if applicable
13 //     }
14
15 //     private void SelectTarget()
16 //     {
17 //         // Logic to select target
18 //     }
19
20 //     private void SelectPosition()
21 //     {
22 //         // Logic to select position
23 //     }
24 // }
```

¿Pero que pasa si simplemente no usamos el código?

```
2 public class InvitationEffect : Effect
3 {
4     private GameState _state;
5     public InvitationEffect(Game State)
6         => _state = state;
7
8     public override void Apply()
9     {
10         SelectTarget();
11         SelectPosition();
12         // Summon and revive unit if applicable
13     }
14
15     private void SelectTarget()
16     {
17         // Logic to select target
18     }
19
20     private void SelectPosition()
21     {
22         // Logic to select position
23     }
24 }
```

A veces los comentarios son superfluos

```
2 public class EffectFactory
3 {
4     public async Effect CreateEffect(SkillData data)
5     {
6         if(data.Type == EffectType.Phys)
7         {
8             //Returns a new DamageEffect with the SkillPower and EffectType.Phys
9             return new DamageEffect(data.SkillPower, EffectType.Phys);
10        }
11        if(data.Type == EffectType.Gun)
12        {
13            //Returns a new DamageEffect with the SkillPower and EffectType.Gun
14            return new DamageEffect(data.SkillPower, EffectType.Gun);
15        }
16        if(data.Type == EffectType.Fire)
17        {
18            //Returns a new DamageEffect with the SkillPower and EffectType.Fire
19            return new DamageEffect(data.SkillPower, EffectType.Fire);
20        }
21        // ... //
22    }
23 }
```

A veces los comentarios son superfluos

```
2 public class EffectFactory
3 {
4     public async Effect CreateEffect(SkillData data)
5     {
6         if(data.Type == EffectType.Phys)
7         {
8
9             return new DamageEffect(data.SkillPower, EffectType.Phys);
10        }
11        if(data.Type == EffectType.Gun)
12        {
13
14            return new DamageEffect(data.SkillPower, EffectType.Gun);
15        }
16        if(data.Type == EffectType.Fire)
17        {
18
19            return new DamageEffect(data.SkillPower, EffectType.Fire);
20        }
21        // ... //
22    }
23 }
```

```

1 private void PrintAllCharacterModifiers(Character character)
2 {
3     var bonus = new List<(string, string, StatModifier)>
4     {
5         ("bonus", character.attack.name, character.attack.bonus),
6         ("bonus", character.speed.name, character.speed.bonus),
7         ("bonus", character.defense.name, character.defense.bonus),
8         ("bonus", character.resistance.name, character.resistance.bonus)
9     };
10    var penalties = new List<(string, string, StatModifier)>
11    {
12        ("penalty", character.attack.name, character.attack.penalty),
13        ("penalty", character.speed.name, character.speed.penalty),
14        ("penalty", character.defense.name, character.defense.penalty),
15        ("penalty", character.resistance.name, character.resistance.penalty)
16    };
17
18    var modifiers = new List<(string type, string name, StatModifier modifier)>();
19    modifiers.AddRange(bonus);
20    modifiers.AddRange(penalties);
21
22    PrintModifiers(character, bonus, AttackType.None);
23    PrintModifiers(character, bonus, AttackType.FirstAttack, " en su primer ataque");
24    PrintModifiers(character, bonus, AttackType.FollowUp, " en su Follow-Up");
25    PrintModifiers(character, penalties, AttackType.None);
26    PrintModifiers(character, penalties, AttackType.FirstAttack, " en su primer ataque");
27    PrintModifiers(character, penalties, AttackType.FollowUp, " en su Follow-Up");
28
29
30    foreach (var stat in modifiers)
31        if (stat.modifier.isNeutralized)
32            _view.WriteLine($"Los {stat.type} de {stat.name} de {character.name} fueron neutralizados");
33 }

```

```

1 private void PrintAllCharacterModifiers(Character character)
2 {
3     // Define los Bonus y Penaltys
4     var bonus = new List<(string, string, StatModifier)>
5     {
6         ("bonus", character.attack.name, character.attack.bonus),
7         ("bonus", character.speed.name, character.speed.bonus),
8         ("bonus", character.defense.name, character.defense.bonus),
9         ("bonus", character.resistance.name, character.resistance.bonus)
10    };
11    var penalties = new List<(string, string, StatModifier)>
12    {
13        ("penalty", character.attack.name, character.attack.penalty),
14        ("penalty", character.speed.name, character.speed.penalty),
15        ("penalty", character.defense.name, character.defense.penalty),
16        ("penalty", character.resistance.name, character.resistance.penalty)
17    };
18    // Añade los Bonus y Penaltys a una lista
19    var modifiers = new List<(string type, string name, StatModifier modifier)>();
20    modifiers.AddRange(bonus);
21    modifiers.AddRange(penalties);
22    // Imprime los Bonus y Penaltys
23    PrintModifiers(character, bonus, AttackType.None);
24    PrintModifiers(character, bonus, AttackType.FirstAttack, " en su primer ataque");
25    PrintModifiers(character, bonus, AttackType.FollowUp, " en su Follow-Up");
26    PrintModifiers(character, penalties, AttackType.None);
27    PrintModifiers(character, penalties, AttackType.FirstAttack, " en su primer ataque");
28    PrintModifiers(character, penalties, AttackType.FollowUp, " en su Follow-Up");
29    // Imprime los Neutralizadores
30    foreach (var stat in modifiers)
31        if (stat.modifier.isNeutralized)
32            _view.WriteLine($"Los {stat.type} de {stat.name} de {character.name} fueron neutralizados");
33 }

```



```

1 private void PrintAllCharacterModifiers(Character character)
2 {
3     // Imprime los bonus
4     var bonus = new List<(string, string, StatModifier)>
5     {
6         ("bonus", character.attack.name, character.attack.bonus),
7         ("bonus", character.speed.name, character.speed.bonus),
8         ("bonus", character.defense.name, character.defense.bonus),
9         ("bonus", character.resistance.name, character.resistance.bonus)
10    };
11    PrintModifiers(character, bonus, AttackType.None);
12    PrintModifiers(character, bonus, AttackType.FirstAttack, " en su primer ataque");
13    PrintModifiers(character, bonus, AttackType.FollowUp, " en su Follow-Up");
14    // Imprime los penalties
15    var penalties = new List<(string, string, StatModifier)>
16    {
17        ("penalty", character.attack.name, character.attack.penalty),
18        ("penalty", character.speed.name, character.speed.penalty),
19        ("penalty", character.defense.name, character.defense.penalty),
20        ("penalty", character.resistance.name, character.resistance.penalty)
21    };
22    PrintModifiers(character, penalties, AttackType.None);
23    PrintModifiers(character, penalties, AttackType.FirstAttack, " en su primer ataque");
24    PrintModifiers(character, penalties, AttackType.FollowUp, " en su Follow-Up");
25    // Imprime los neutralizadores
26    var modifiers = new List<(string type, string name, StatModifier modifier)>();
27    modifiers.AddRange(bonus);
28    modifiers.AddRange(penalties);
29    foreach (var stat in modifiers)
30        if (stat.modifier.isNeutralized)
31            _view.WriteLine($"Los {stat.type} de {stat.name} de {character.name} fueron neutralizados");
32 }

```

Agregando algunos métodos, la función anterior se reduce a esto:

```
1 private void PrintAllCharacterModifiers(Character character)
2 {
3     // Imprime bonus
4     var bonus = GetBonusInfoByStat(character);
5     PrintModifiers(character, bonus);
6
7     // Imprime penalties
8     var penalties = GetPenaltiesInfoByStat(character);
9     PrintModifiers(character, penalties);
10
11    // Imprime neutralizadores
12    var modifiers = Concatenate(bonus, penalties);
13    PrintNeutralizers(character, modifiers);
14 }
```

Agregando algunos métodos, la función anterior se reduce a esto:

```
1 private void PrintAllCharacterModifiers(Character character)
2 {
3     // Imprime bonus
4     var bonus = GetBonusInfoByStat(character);
5     PrintModifiers(character, bonus);
6
7     // Imprime penalties
8     var penalties = GetPenaltiesInfoByStat(character);
9     PrintModifiers(character, penalties);
10
11    // Imprime neutralizadores
12    var modifiers = Concatenate(bonus, penalties);
13    PrintNeutralizers(character, modifiers);
14 }
```

... y podemos seguir limpiando esto

¿Cuáles son los descuentos típicos en el proyecto debido al cap. 4?

Descuentos más comunes:

- Código sucio sin comentar.
- Comentarios innecesarios.
- Bloques de código comentado.
- Comentarios que entorpezcan la lectura del código.

Descuentos más comunes:

- Código sucio sin comentar.
- Comentarios innecesarios.
- Bloques de código comentado.
- Comentarios que entorpezcan la lectura del código.

Importante: No basta con borrar todos tus comentarios antes de una entrega. Hay que limpiar el código hasta que los comentarios se vuelvan innecesarios.

Actividad

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

En simple, existen usuarios. Cada usuario tiene varios clientes. Cada cliente tiene una carpeta “depot” en el servidor donde puede crear, modificar y borrar archivos.

Interrogación 2024-1

Perforce es un sistema de control de versiones que permite gestionar el desarrollo de software mediante la organización y el control de cambios en el código fuente. Es parecido a Git pero centralizado.

Cada usuario en Perforce puede tener varios clientes. Y cada cliente tiene un depot. El depot es una ruta a una carpeta dentro del servidor. El usuario podrá extraer y modificar todos los archivos dentro de su depot (incluyendo, crear carpetas).

En simple, existen usuarios. Cada usuario tiene varios clientes. Cada cliente tiene una carpeta “depot” en el servidor donde puede crear, modificar y borrar archivos.

Para crear/modificar/borrar archivos les damos una librería:

- `void Add(FilePath filePath, string? content).`
- `bool Exists(FilePath path).`
- `Files Get(FilePath path, string pathToFile).`

El proyecto *Perforce* es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

El proyecto *Perforce* es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

`Controller.cs` tiene un método de entrada que recibe un `Request` con información asociada a lo que el usuario quiere hacer. En particular,

- Si `request.Type` es “client”, se crea un cliente o se obtiene su información.
- Si `request.Type` es “sync”, se obtiene información de los archivo(s) en el depot.
- Si `request.Type` es “submit”, se suben cambios al servidor. Esta solicitud está relacionada con una lista de cambios, los cuales pueden ser agregar, editar o eliminar archivos. Se espera soportar más tipos de cambio en el futuro.

El proyecto *Perforce* es una primera versión de un servidor de *Perforce*. Su clase de entrada es `Controller.cs` y gestiona tanto a los clientes como a sus archivos.

`Controller.cs` tiene un método de entrada que recibe un `Request` con información asociada a lo que el usuario quiere hacer. En particular,

- Si `request.Type` es “`client`”, se crea un cliente o se obtiene su información.
- Si `request.Type` es “`sync`”, se obtiene información de los archivo(s) en el depot.
- Si `request.Type` es “`submit`”, se suben cambios al servidor. Esta solicitud está relacionada con una lista de cambios, los cuales pueden ser agregar, editar o eliminar archivos. Se espera soportar más tipos de cambio en el futuro.

`Handle(...)` retorna un objeto de tipo `Response` con información sobre la ejecución de la `Request` (e.j., si la *request* fue exitosa, la información solicitada, etc).